

Event Handling in JS

1. What are Events?

- a. Event is an object in JavaScript.
- b. The change in the state of an object is known as an *Event*.
- c. When an event occurs, it triggers a JavaScript function.
- d. Events let us make web pages interactive by responding to user actions.
- e. *Examples* → clicking a button, pressing a key, submitting a form, scrolling a page.

```
<button onclick="alert('Hello Events')">Click Me</button>
```

2. What is `addEventListener()`?

- a. The `addEventListener()` method attaches an event handler or event listener to HTML elements.
- b. The `addEventListener()` is an inbuilt javascript function.
- c. **The `addEventListener()` method takes two arguments:**
 - i. The name of the event
 - ii. The function that you want to be called when the event occurs. (callback function). By default the callback function will accept the *event object (e)* as a parameter.

d. Syntax →

```
element.addEventListener("event_type", callback_function)
```

- e. **For Example** → *When you click anywhere on the window screen it will return the `PointerEvent`.*

```
window.addEventListener("click", (e) => {  
    console.log("Clicked on the screen!");  
});
```

- f. **For Example** → When you click on the button it will return the current targeted element with the help of **currentTarget** property. (Here, I am clicking on the button that's why it is returning the button element)

```
<button id="btn">Click Me</button>
let btn = document.getElementById("btn");
btn.addEventListener("click", (e) => {
  console.log(e.currentTarget);
});
Output: <button id="btn">Click Me</button>
```

3. What is **PointerEvent**?

- a. PointerEvent is a **special event** object in JavaScript that represents interactions made with any pointing device.

- b. **For Example** →

```
<button id="btn">Click Me</button>
let btn = document.getElementById("btn");
btn.addEventListener("click", (e) => {
  console.log(e); // PointerEvent Object
});
```

4. DOM Events Types:

- a. The events are categorized based on the type of interaction they represent.
- b. Below are the main event categories in the DOM:
- i. Mouse Events
 - ii. Keyboard Events
 - iii. Form Events
 - iv. Focus Events

- v. Window Events
- vi. Clipboard Events

5. Mouse Events:

- a. **Mouse events** are triggered by mouse actions, such as clicking, hovering, or moving the mouse.
- b. **Below are the main events which comes under the mouse event category:**
 - i. **click** → A click event occurs when you press and release a mouse button on an element, like a button.
 - ii. **dblclick** → A double-click event occurs when you quickly click the mouse button twice on an element.
 - iii. **mousedown** → This event trigger when you press down the mouse button over an element.
 - iv. **mouseup** → This event trigger when you release a mouse button after pressing it.
 - v. **mousemove** → This event is triggered whenever the mouse moves over an element.
 - vi. **mouseenter** → This event trigger when the mouse pointer enters the area of an element (it doesn't trigger when entering child elements).
 - vii. **mouseleave** → This event occurs when the mouse pointer leaves the boundary of an element.
 - viii. **mouseover** → This event is triggered when the mouse pointer enters an element or any of its child elements.
 - ix. **mouseout** → This event occurs when the mouse pointer leaves the area of an element or any of its child elements.

6. Keyboard Events:

- a. **Keyboard events** allow you to capture key presses, key releases, and other keyboard-related actions on an HTML page.
- b. **There are three main types of keyboard events in the DOM:**
 - i. **keydown** → This event fires when a key is pressed down, before the character is generated
 - ii. **Note:** To capture which key the user has pressed, you can access the **key** or **keyCode** property of the Keyboard Event.
 - iii. **keypress (deprecated)** → This event is fired when a key is pressed and a character is generated (such as typing a letter or number). It is now deprecated in favor of keydown and keyup .
 - iv. **Keyup** → This event occurs when you release a key after pressing it. It is typically used to detect when the user has finished pressing a key.

7. Form Events:

- a. **Form events** in the DOM are events related to form elements like `<input>`, `<textarea>` , `<select>` , `<button>` , and `<form>`.
- b. **There are five main form events are as below mentioned:**
 - i. **submit** → This event is triggered when a form is submitted, typically when the submit button is clicked or the enter key is pressed within an input field.

- ii. *change* → This event is fired when a change is made to an input element.
- iii. *focus* → This event occurs when a user clicks in a form input field. (That means we are focusing the input field)
- iv. *blur* → This event occurs when the form element loses focus, such as when the user clicks away or presses the Tab key to move to another input.
- v. *input* → This event is triggered immediately when the value of a form element changes (e.g., while typing in an input field). It is typically used for real-time input validation or dynamically updating the user interface based on user input.

8. Focus Events:

- a. *Focus events* in the DOM are used to track when an element (like an input field or a form element) gains or loses focus.
- b. **There are two main focus events are there:**
 - i. *focus*
 - ii. *blur*
- c. The same use case takes reference from above *form events* focus and blur.

9. Window Events:

- a. *Window events* are events related to the overall window or document state, such as when the page is loaded, resized, or scrolled.
- b. These events directly provide the *window* object.

c. There are main 4 window events are as follows:

- i. **load** → This event fires when the entire page, including all assets (images, scripts, and stylesheets), has finished loading.
- ii. **resize** → The resize event is triggered when the window is resized (either by the user or through a script). It is commonly used in responsive web design to adjust layouts based on the new window size.
- iii. **scroll** → The scroll event is fired when the user scrolls the page (either vertically or horizontally).
- iv. **beforeunload** → The beforeunload event is fired when the user is about to navigate away from the page or close the browser. It's used to warn users about unsaved changes or any other important information before they leave the page

For Example →

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>BeforeUnload Example</title>
  </head>
  <body>
    <h2>BeforeUnload Event Example</h2>
    <p>
      Type something in the box. If you try to refresh or leave the page, you'll
      get a warning!
    </p>

    <textarea
      id="myInput"
      rows="5"
      cols="40"
      placeholder="Type here..."
    ></textarea>

    <script>
      let isChanged = false;

      // Mark as changed when user types
      document.getElementById("myInput").addEventListener("input", () => {
        isChanged = true;
      });
```

```

// Warn user before leaving
window.addEventListener("beforeunload", (e) => {
  const message = "Are you sure you want to leave?";
  e.returnValue = message; // modern browsers ignore this message
  return message; // for older browsers
});
</script>
</body>
</html>

```

10. Clipboard Events:

- a. *Clipboard events* are triggered when interacting with the clipboard (copying, cutting, and pasting content).
- b. **Following are the three main clipboard events:**
 - i. *copy* → Triggered when the user copies content to the clipboard.
 - ii. *cut* → Triggered when the user cuts content to the clipboard.
 - iii. *paste* → Triggered when the user pastes content from the clipboard.

11. What is `e.preventDefault()`?

- a. The `e.preventDefault()` method is used to prevent the default behavior of an event from occurring.
- b. It is basically used to prevent the default behaviour after submitting the form page will reload or refresh or to click on anchor tag.
- c. **Syntax** → `e.preventDefault()`; // Here, e stands for event
- d. **Example** → **Form Submission**

```

<form id="myForm">
  <input type="text" placeholder="Enter something required" />
  <button type="submit">Submit</button>
</form>

<script>
  document.getElementById("myForm").addEventListener("submit", (e) => {
    e.preventDefault(); // stop default form submission (page reload)
  });

```

```
        alert("Form submitted using JavaScript!");
    });
</script>
```

e. Example → Anchor Link

```
<a href="https://www.google.com/" id="myLink">Go to Google</a>
```

```
<script>
    document.getElementById("myLink").addEventListener("click", (e) => {
        e.preventDefault(); // stop navigation
        alert("Navigation prevented. Staying on this page!");
    });
</script>
```

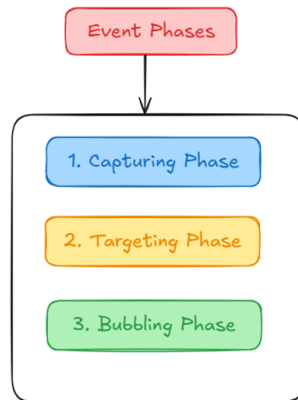
12. DOM Event Propagation Phases:

a. What is Event Propagation?

- i. Event Propagation means Movement of Events through the DOM.
- ii. It will travel through the DOM when an event occurs and reaches to the target element, triggering further actions based on the event.
- iii. It happens in three phases Capturing, Targeting and Bubbling

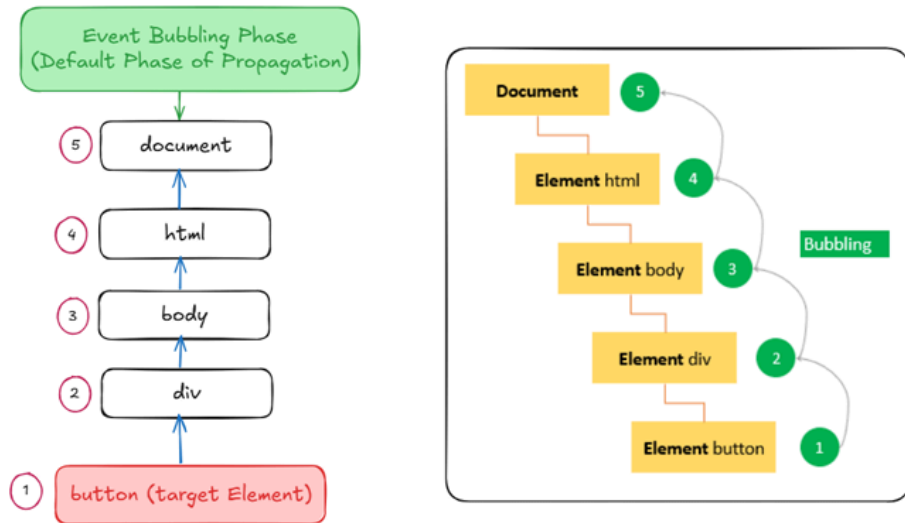
b. Event Phases Event Flow):

- i. Event Bubbling
- ii. Event Capturing



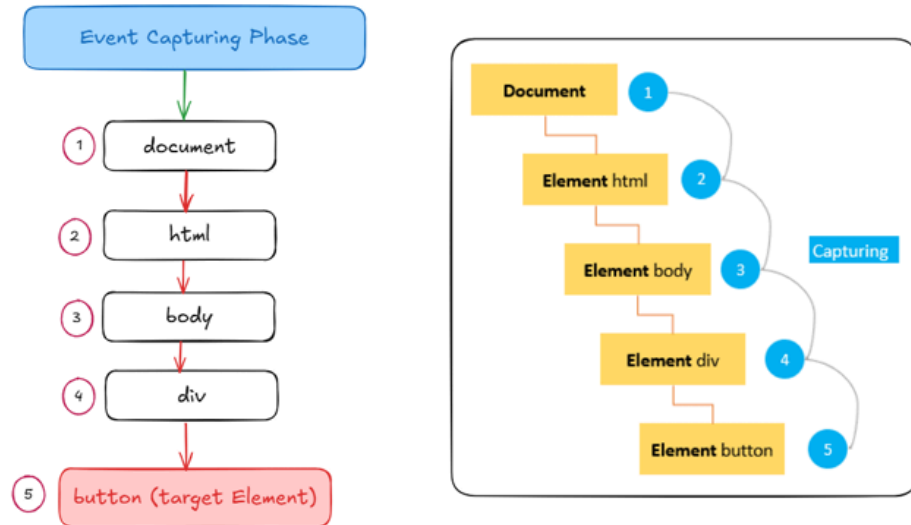
c. Event Bubbling Phase ⇒

- i. In the event bubbling model, an event starts at the most specific element (target element) and then flows upward toward the least specific element (root element) (the document or even window).
- ii. When you click the button, the click event occurs in the following order.
 1. button
 2. div with the id container
 3. body
 4. html
 5. document
- iii. The following picture illustrates the event bubbling effect when users click the button:



d. Event Capturing Phase ⇒

- i. In the event-capturing model, an event starts at the least specific element (root element) and flows downward toward the most specific element (target element).
- ii. When you click the button, the click event occurs in the following order:
 1. document
 2. html
 3. body
 4. div with the id container
 5. button
- iii. The following picture illustrates the event-capturing effect:



e. DOM Level 2 Event Flow:

- DOM level 2 events specify that event flow has three phases:
 - First, event capturing occurs, which provides the opportunity to intercept the event.
 - Then, the actual target receives the event.
 - Finally, event bubbling occurs, which allows a final response to the event.
- The following picture illustrates the DOM Level 2 event model when users click the button:

